

Reinforcement Learning of (Stylised) Blackjack

Kavya Vijaysankar (20794888), Ceren Kimyonok (20794748), and Leslie Belem Quintana Lopez
(20796978)

Machine Learning in Science (PHYS4035)
School of Physics and Astronomy, University of Nottingham

February 2026

Abstract

This project investigates the training and evaluates the performance of a Reinforcement Learning agent in a stylized version of Blackjack, utilising tabular Q-learning, and focusing on the agent's ability to navigate both stationary and non-stationary stochastic environments. We examine an infinite-deck environment characterized by independent and identically distributed card draws, and a finite-deck environment where sampling without replacement introduces temporal dependencies. To handle the non-stationary nature of the finite-deck environment, we augment the state space with discretized features that track the deck composition, allowing the process to be treated as approximately Markovian. Furthermore, we utilize optimistic initial values in the finite-deck agent to facilitate efficient additional exploration across the expanded state space. Performance is benchmarked against heuristic baselines to compare training outcomes across both environments.

1 Introduction

This project investigates the application of Reinforcement Learning (RL) to a stylised version of Blackjack. Unlike the standard casino version, this game is simplified to a single-player scenario where the dealer is passive. The primary objective is to maximize the accumulated score over an episode.

This paper begins with an overview of the rules of Stylised Blackjack and introduces the two environments and their respective definitions of an episode. It then describes the chosen reinforcement learning algorithm and the corresponding environment and agent architectures. Finally, the training procedures for both environments are presented, followed by an analysis of the training dynamics and an evaluation of agent performance.

1.1 Rules of Stylised Blackjack

The game is played with one or more standard 52-card decks. Card values follow standard Blackjack conventions: numbered cards (2–10) take their face value, face cards (J, Q, K) are valued at 10, and Aces are valued at 11 unless the hand total exceeds 21, in which case they are valued at 1. The suit of the cards is irrelevant. Each hand begins with the agent receiving a single card, followed by iterative hit or stick decisions until the hand terminates upon sticking or reaching a value ≥ 21 . The terminal reward assigned at the end of a hand is defined as

$$S = \begin{cases} \left(\sum_{i=1}^{n+1} C_i \right)^2 & \text{if } \sum_{i=1}^{n+1} C_i \leq 21 \\ 0 & \text{if } \sum_{i=1}^{n+1} C_i > 21 \end{cases} \quad (1)$$

where C_i denotes the numerical value of the i th card in the hand.

2 RL Model

The reinforcement learning task is formalized as a Markov Decision Process (MDP) defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R)$. The state space $\mathcal{S}$ is defined by a feature vector that scales with the environment’s complexity (discussed further in sections 3.1 and 3.2). The action set $\mathcal{A}$ consists of the choices {Hit, Stick}.

The core algorithm used is Q-Learning, an off-policy, model-free method that seeks to find the optimal q-function through experience. The agent updates its knowledge (Q-values) using the following formula:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[R + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (2)$$

where $Q(s, a)$ is the current estimate of the value of taking action a in state s , α is the learning rate which controls how much new information overrides old information, R is the immediate reward received after taking

action a , γ is the Discount Factor which determines the importance of future rewards, and $\max_{a'} Q(s', a')$ is the estimate of the optimal future value from the next state s' .

3 Environment & Agent Architectures

The episode structure varies significantly between the two deck environments: `infinite.env.py` treats each hand as an independent stationary episode with no historical dependencies. On the other hand, `finite.env.py` defines an episode as the sequence of hands until the D decks are exhausted, introducing temporal dependencies as the underlying probability distribution shifts as the deck depletes.

3.1 Infinite Deck Design

As implemented in `infinite.env.py`, the state space $\mathcal{S}$ is defined by the tuple $(\text{player_sum}, \text{usable_ace})$, representing the current total and the flexibility of an Ace’s value. Because the deck is infinite, the probability of drawing any specific card value is constant.

3.2 Finite Deck Design

To account for the non-stationary transition dynamics induced by deck depletion, `finite.env.py` utilizes an augmented state representation to approximate the latent deck state. The system incorporates the following features in addition to `player\_sum` and `usable\_ace`:

- **running_count**: A tracking variable that updates based on the value of observed cards, serving as a heuristic for the composition of the remaining deck.
- **deck_depth**: The number of cards remaining, which indicates how much the odds of drawing certain cards have shifted as the deck depletes.
- **true_count**: A normalized metric calculated by dividing the **running_count** by the number of remaining decks; this serves as a sufficient statistic for the conditional probability of drawing high-value cards.

To ensure the Q-table remains at a manageable size for training, these continuous metrics are mapped to **3 bins**. This reduces the total number of possible states while retaining the statistical information the agent needs to adapt its strategy as the deck composition changes.

3.3 Agent Implementation

The learning logic within the `agent.py` file is divided into 2 classes: `BlackjackAgent` and `Agent\_finite\_deck`. Both agent classes implement the

tabular Q-learning algorithm and utilize an ϵ -greedy exploration policy, but are configured with different initialization strategies to suit their respective environments. To handle the expanded state space of the finite-deck environment, we employed optimistic initial values by initializing the Hit action to 500.0. This biased initialization acts as an intrinsic exploration mechanism, ensuring that high-risk actions are explored adequately and reducing the risk of premature convergence to a sub-optimal strategy.

4 Infinite Deck Training

The training for the infinite deck environment was conducted over 900,000 episodes. Given that in this environment card draw probabilities remain constant at each step, the agent focused on discovering the optimal mapping between the state space and the action set.

The model utilized a learning rate of $\alpha = 0.01$ and a discount factor of $\gamma = 0.9$. To balance discovery and optimization, we implemented an ϵ -greedy decay strategy starting at 1.0 and reaching a minimum exploration floor of 0.0067. The **Learning Curve** as shown in the Figure 1 (a) illustrates the moving average of rewards. We can observe a steady growth from an initial 173.47 to a stable plateau of approximately 283.04.

5 Finite Deck Training

For the finite deck environment, the agent was trained over 1,000,000 episodes and used a randomized deck number configuration where the number of decks, D , was sampled such that $D \in [1, 8]$ for each episode. The upper bound of eight was chosen to align with industry standards, as casinos typically use 6 or 8 decks. Varying the number of decks during training prevented the agent from over-fitting to a specific deck depth, forcing it to rely on the normalized `true_count` and `deck_depth` features rather than absolute card counts.

The model used a learning rate of $\alpha = 0.01$ and a discount factor of $\gamma = 0.98$. We implemented a dynamic decay rate to ensure that the agent remained in an exploratory phase for 99% of the duration of the training. The decay factor was calculated by

$$\epsilon_{decay} = \left(\frac{\epsilon_{min}}{\epsilon_{start}} \right)^{\frac{1}{T_{target}}} \quad (3)$$

where T_{target} is $0.99 \times \text{Total Episodes}$. This ensures that the agent only reaches its minimum exploration towards the end of the training.

Figure. 1 (b), which illustrates the **Learning Curve**, shows a moving average of rewards normalized by deck

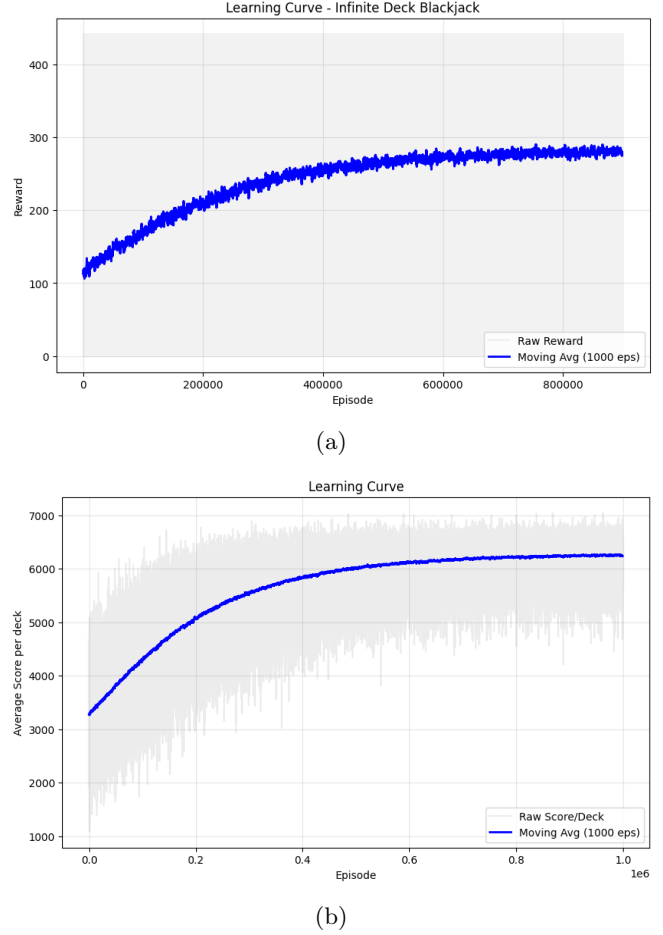


Figure 1: The agent has successfully moved beyond a random policy and the Q-table has approximated the optimal value function

count, showing a clear upward trajectory.

By also monitoring the average number of hands played per deck, we can see a transition in the hitting policy, optimised for the quadratic reward structure. This metric decreased from an initial 29 to a stable plateau of approximately 23–24.

6 Performance Evaluation

The performance of both the infinite- and finite-deck agents was evaluated against a fixed, non-adaptive heuristic baseline that follows a stationary rule, making decisions solely based on the current hand total. The baseline implements a deterministic stand-on-17 strategy in which the agent continues to draw cards until the hand total reaches at least 17, after which it sticks.

To mitigate the effects of environmental stochasticity, performance for both the baseline and the agent models was established over 50,000 episodes, ensuring a stable reference for comparative analysis. The evaluation was conducted with ϵ set to 0, ensuring the model utilized its

learned greedy policy without further exploration.

6.1 Infinite Deck Agent Performance

While the baseline achieved a total net reward of 13,017,811, the trained agent reached 14,111,210. The agent maintained a controlled Bust Rate of 3.46%, optimizing the balance between risk and the quadratic reward structure. Figure 2 shows the score distribution, where the agent’s outcomes are shifted toward higher rewards, indicating fewer busts.

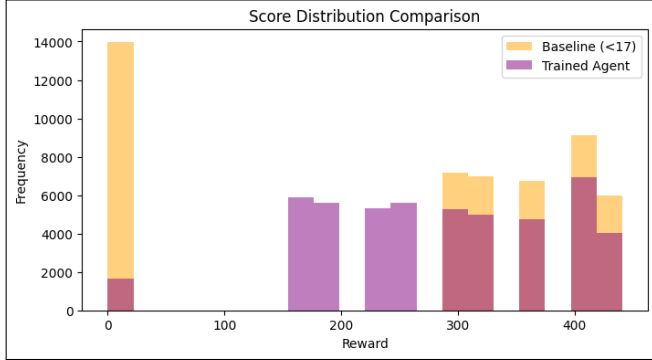


Figure 2: Score Distribution (50,000 Episodes)

By inspecting the final Q-table, we gain insight into the specific behaviors that drive this performance lift. The agent learned a more nuanced strategy compared to the baseline as detailed in Table 1. The agent learned to prioritize risk mitigation in Hard hands (Sum 13-16), choosing to *Stick* where the baseline would aggressively *Hit*. In contrast, in soft hands (With Ace), the agent uses the flexibility of Ace to pursue higher totals.

Table 1: Policy Divergence: Baseline vs. Agent

Hand	Action	AI (No Ace)	AI (Ace)	Notes
11–12	HIT	HIT	HIT	Agreement
13–15	HIT	STICK	HIT	Risk mitigation for hard hands
16	HIT	STICK	STICK	Conservative stand choice
17–21	STICK	STICK	STICK	Agreement

6.2 Finite Deck Agent Performance

The evaluation aims to assess the agent can exploit shifting card probabilities and defeat the baseline logic. Figure 3 shows the score distribution for the evaluation loop using a single deck. The distribution is unimodal and approximately Gaussian, indicating stable and consistent

performance across episodes, with relatively low variance. Table 1 presents a comparative analysis of the av-

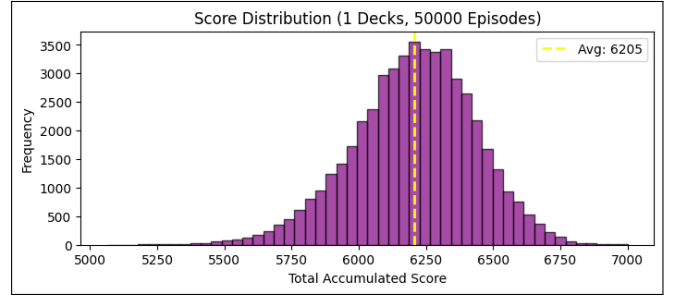


Figure 3: Score Distribution (1 Deck, 50,000 Episodes)

erage performance metrics for the reinforcement learning agent and the stationary baseline for deck sizes ranging from 1 to 8. The agent achieved a 35% improvement in mean score compared to the baseline, indicating that it effectively tracks changes in card probabilities.

Deck size	Baseline Average	Agent Average
1	4,577	6,205
2	9,299	12,517
3	13,878	18,834
4	18,437	25,184
5	23,307	31,537
6	27,763	37,745
7	32,638	43,873
8	37,278	49,525

Table 2: Baseline VS Agent Performance

7 Conclusion

This project shows that a tabular Q-learning reinforcement learning agent can learn effective policies in both infinite and finite deck Blackjack environments. In the infinite-deck setting, the agent converges to a stable strategy aligned with the reward structure, while in the finite-deck setting, it uses augmented state information to adapt its decisions as the deck composition changes. In both cases, the learned policies outperform the heuristic baseline, demonstrating that the agent exploits environment-specific structure.

References

- [1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.
- [2] E. O. Thorp, *Beat the Dealer: A Winning Strategy for the Game of Twenty-One*, Random House, 1966.
- [3] C. J. C. H. Watkins and P. Dayan, “Q-learning,” *Machine Learning*, vol. 8, pp. 279–292, 1992.